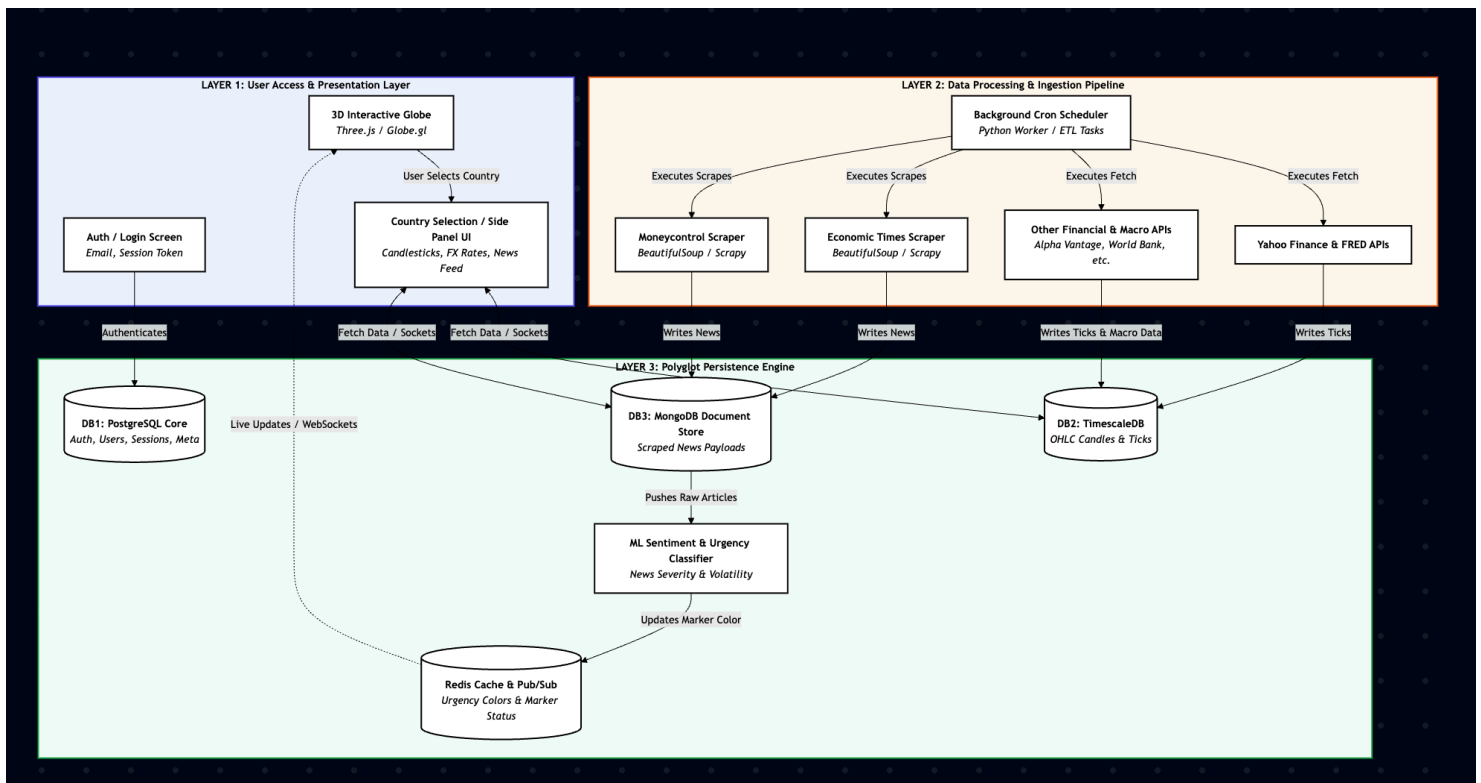


Chrono

Ideation



Auth/Data Ingestion/Data Updation Pipeline

Layer 1: User Access & Presentation Layer

- **Authentication Gateway & Access Control:**
Access to the application is managed through a Single Page Application (SPA) authentication flow utilizing JWTs or HTTP-only session cookies. Client-side route guards verify token validity on page load, keeping unauthenticated users on the login interface while granting authenticated sessions access to the 3D canvas. Credentials, user profiles, and session states are verified and managed directly against **PostgreSQL (DB1)**.
- **3D Interactive Spatial Canvas (Globe.gl / Three.js):**
The landing page centers on a full-screen, high-performance 3D globe built using **Globe.gl** and **Three.js** running on WebGL shaders. Geographic country boundaries are rendered using GeoJSON polygon datasets, while key stock exchanges and financial hubs are highlighted with 3D instanced mesh markers. The spatial canvas listens to incoming WebSocket events to dynamically update marker colors based on real-time urgency scores without requiring canvas re-renders.

- **3D Raycasting & Smooth Navigation:**
When a user hovers over or clicks on a country, 3D raycasting detects the mouse intersection coordinates relative to the 3D sphere and resolves the corresponding ISO country code. This interaction triggers a smooth camera interpolation (zooming and rotating the camera toward the selected nation's coordinates) while simultaneously sliding open the localized financial control panel.
- **Localized Financial Control Panel & Charting UI:**
The side drawer functions as a dedicated dashboard for the selected nation, rendering modular visualization components for micro- and macroeconomic indicators. It uses high-performance charting libraries (such as TradingView Lightweight Charts or Plotly.js) to display OHLC candlestick charts, moving averages, FX volatility, and market depth, while presenting local commodity prices and scraped news feeds in structured widgets.
- **Hybrid Data Fetching & Real-Time Streaming:**
The panel operates on a dual data-retrieval pipeline: standard REST or GraphQL API endpoints execute targeted SQL queries to pull structured historical time-series data from **TimescaleDB (DB2)** and country metadata from **PostgreSQL (DB1)**. Concurrently, an persistent WebSocket connection listens to **Redis Pub/Sub** to stream real-time price tick updates, breaking news payloads from **MongoDB (DB3)**, and dynamic market sentiment classifications directly to the UI.

Layer 2: Data Processing & Ingestion Pipeline

- **Background Cron Scheduler & Task Orchestration:** The ingestion layer relies on an automated background execution engine (such as Python's **APScheduler**, **Celery**, or native **asyncio** task loops). This scheduler runs on configured time intervals, decouples external data fetching from the frontend UI, and prevents rate-limit bottlenecks on external providers while serving as an automated ETL (Extract, Transform, Load) pipeline.
- **Structured Financial & Macro API Connectors:** Dedicated API worker modules execute periodic HTTP GET requests to structured financial services (such as Yahoo Finance, FRED, Alpha Vantage, and World Bank APIs). These connectors pull high-frequency market ticks, daily OHLC (Open, High, Low, Close) candlestick prices, foreign exchange rates, commodity benchmarks, and global macroeconomic indicators (like GDP, CPI inflation, and interest rates).
- **Unstructured Web Scraping Engines:** Automated scraping scripts (built with libraries like **BeautifulSoup**, **Scrapy**, or **Playwright**) continuously crawl major news portals (such as Economic Times, Moneycontrol, and Reuters). These scrapers extract raw, unstructured editorial data—including breaking news headlines, article text, publication timestamps, and regional tags—that financial APIs often restrict or paywall.
- **ETL Data Parsing & Transformation:** Before writing to persistence layers, raw payloads are passed through a Python transformation pipeline. This pipeline cleans incoming JSON and HTML, normalizes currency units, converts timestamp formats into UTC time-series standards, and tags

unstructured news stories with standardized ISO country codes to match them with the global globe map.

- **Targeted Database Persistence Routing:** Once transformed, the pipeline routes data to its corresponding database based on structure and access patterns: normalized time-series price ticks are written via bulk inserts into **TimescaleDB (DB2)**, while unstructured news documents and raw scraped payloads are persisted into **MongoDB (DB3)**.

Layer 3: Polyglot Persistence Engine

- **PostgreSQL Core (DB1 — Relational Store):** Serves as the central ACID-compliant relational authority for system administration, identity management, and static reference mapping. It stores user accounts, hashed credentials, active session tokens, subscription tiers, and core geographical metadata (such as ISO country codes, capital city coordinates, central bank profiles, and stock exchange directories).
- **TimescaleDB Time-Series Engine (DB2 — Financial Metrics Store):** Functions as a specialized PostgreSQL extension engineered to manage write-heavy, high-frequency time-series datasets. Organized into partitioned "hypertables," it stores OHLC candlestick prices, intraday market ticks, commodity benchmarks, and foreign exchange rates. It utilizes continuous aggregates to automatically calculate moving averages, volatility metrics, and historical index trends directly inside the database, optimizing read queries for the charting UI.
- **MongoDB Document Store (DB3 — Unstructured Data Store):** Acts as a flexible, schema-less document repository dedicated to persisting raw and unstructured editorial content. It ingests scraped financial news stories, breaking press releases, and raw API JSON payloads. Utilizing BSON document structures and full-text search indexes, MongoDB enables fast keyword searching and metadata filtering by ISO country code and publishing timestamp.
- **Machine Learning Sentiment & Urgency Classifier:** Operates as an asynchronous analytical service listening to MongoDB document inserts. As new articles hit DB3, the ML model processes the unstructured text to evaluate sentiment (Bullish, Bearish, Neutral) and assign an urgency/impact score based on market volatility potential or political economic stress.
- **Redis In-Memory Cache & Pub/Sub Broker:** Provides an in-memory caching and messaging layer to bridge database updates with real-time frontend rendering. It caches calculated country status flags and urgency scores to eliminate repetitive database reads. Additionally, its Pub/Sub mechanism intercepts high-priority classification tags from the ML agent and instantly pushes color-code updates to the 3D globe via WebSocket channels.

Database Details (PPT Purposes)

Here is the technical rationale and breakdown for each database in your polyglot persistence architecture, structured specifically for your project documentation or presentation.

DB1: PostgreSQL (Relational Core)

- **Purpose & What It Is Doing:**
 - **Role:** Serves as the system of record for structured, highly relational core data.
 - **Tasks:** Manages user authentication, hashed credentials, active session tokens, user preferences (e.g., custom watchlists, bookmarked countries), and master reference mapping (ISO country codes, central bank details, stock exchange directories).
- **Why We Chose It:**
 - **ACID Compliance & Strict Consistency:** Authentication and user states cannot tolerate data drift, lost updates, or eventual consistency issues. PostgreSQL guarantees full ACID (Atomicity, Consistency, Isolation, Durability) properties.
 - **Relational Integrity:** Foreign key constraints (**ON DELETE CASCADE**, strict schema enforcement) prevent orphaned records when users modify custom alert rules or watchlists.
 - **Ecosystem Alignment with DB2:** TimescaleDB (DB2) is built directly on top of PostgreSQL. Using PostgreSQL for DB1 allows your backend to share connection pools, ORMs (e.g., SQLAlchemy/Prisma), and database drivers (**psycopg2/asyncpg**) across both DB1 and DB2.
- **Why We Didn't Choose Something Else:**
 - **Why Not MongoDB?** Storing relational, deeply nested user preferences or permissions in a document store requires expensive **\$lookup** operations (manual joins) and risks data duplication across documents.
 - **Why Not Redis?** Redis lives in memory. Using it as a primary persistent database for user accounts is cost-inefficient and lacks the complex relational querying capabilities needed for user management.

DB2: TimescaleDB (Time-Series Engine)

- **Purpose & What It Is Doing:**
 - **Role:** Serves as the high-throughput persistence layer for continuous, ordered, time-stamped market data.
 - **Tasks:** Stores high-frequency intraday market ticks, daily OHLC (Open, High, Low, Close) candlestick prices, foreign exchange benchmark rates, and commodity price histories.
- **Why We Chose It:**
 - **Time-Series Partitioning ("Hypertables"):** TimescaleDB automatically partitions incoming data into time/space chunks (e.g., 1-day or 1-week intervals). This keeps index sizes small and maintains high-speed write performance even as your dataset grows into millions of rows.

- **Continuous Aggregates:** Calculates financial indicators (like 50-day moving averages or hourly price volume metrics) automatically in the background using native SQL continuous aggregates, rather than forcing your Python backend to calculate them on the fly.
 - **Native SQL Compatibility:** You get standard SQL analytical functions (`time_bucket()`, window functions) while maintaining full join compatibility with your PostgreSQL metadata tables in DB1.
- **Why We Didn't Choose Something Else:**
 - **Why Not Standard PostgreSQL?** Standard relational tables hit a performance wall with high-frequency time-series data; as B-Tree indexes grow, insert speeds degrade significantly.
 - **Why Not InfluxDB?** While InfluxDB is a solid pure time-series database, it uses its own query language (Flux/InfluxQL) and cannot easily join financial ticks directly with relational SQL tables in DB1.

DB3: MongoDB (Document Store)

- **Purpose & What It Is Doing:**
 - **Role:** Serves as a schema-less repository for unstructured, variable-length text and raw API responses.
 - **Tasks:** Ingests scraped news articles (headline, body text, author, publisher tags) from sites like Moneycontrol and Economic Times, as well as raw JSON payloads fetched from third-party APIs.
- **Why We Chose It:**
 - **Polymorphic / Flexible Schema:** Web scrapers and external news APIs constantly change format, return missing fields, or include varying metadata payloads. MongoDB handles JSON/BSON natively without requiring rigid schema migrations every time a news provider updates its structure.
 - **Unstructured Text & Indexing:** Provides built-in text search indexing and geo/spatial query support, making it simple to query raw news feeds by ISO country code, keywords, or publication date before passing them to the ML model.
- **Why We Didn't Choose Something Else:**
 - **Why Not PostgreSQL JSONB?** While PostgreSQL supports JSON columns, running heavy full-text operations and high-volume, un-normalized scraping writes on DB1 would compete for resources with critical authentication and session queries.
 - **Why Not Cassandra?** Cassandra is built for massive horizontal write scaling across distributed clusters, but it requires fixed schemas and rigid query patterns, making it unsuitable for varied, unstructured news scraping.

In-Memory Layer: Redis (Cache & Pub/Sub Broker)

- **Purpose & What It Is Doing:**
 - **Role:** Functions as a fast, transient, in-memory caching layer and real-time messaging pipeline.
 - **Tasks:** Caches calculated country urgency status flags (so the frontend doesn't repeatedly query DB3) and acts as a Pub/Sub message broker to push real-time ML sentiment updates and dynamic marker color changes to the UI via WebSockets.

- **Why We Chose It:**
 - **Sub-Millisecond Read Latency:** Serving the current color/urgency state of every nation directly from memory guarantees fast globe updates when users land on the page.
 - **Native Pub/Sub Engine:** Redis contains a light message broker. When the ML agent classifies an article, it publishes an event to a Redis channel, which immediately pushes the update over active WebSockets to the frontend without touching disk storage.
- **Why We Didn't Choose Something Else:**
 - **Why Not RabbitMQ or Apache Kafka?** Kafka and RabbitMQ are powerful message brokers, but they do not provide key-value caching. Redis solves both key-value status caching and WebSocket event pub/sub in a single container.
 - **Why Not In-Memory Python Dicts?** Keeping state in Python memory locks that state to a single backend process. If you scale your API across multiple worker instances, a shared Redis instance is required so all workers read the same state.
 -

Summary Table for Quick Reference

Storage Engine	Data Type	Primary Role	Core Reason for Choice
DB1: PostgreSQL	Structured / Relational	Auth, Users, Sessions, Master Metadata	Strict ACID compliance, relational integrity, shared ecosystem with TimescaleDB.
DB2: TimescaleDB	Structured Time-Series	High-frequency OHLC ticks, Candles, FX rates	Automatic time-based partitioning ("hypertables"), native SQL continuous aggregates.
DB3: MongoDB	Unstructured Documents	Scraped news stories, raw API payloads	Flexible schema-less BSON structure, fast text indexing.
Redis	In-Memory Key-Value / Streams	Urgency status cache, WebSocket Pub/Sub	Sub-millisecond latency, built-in Pub/Sub broker to drive live UI updates.